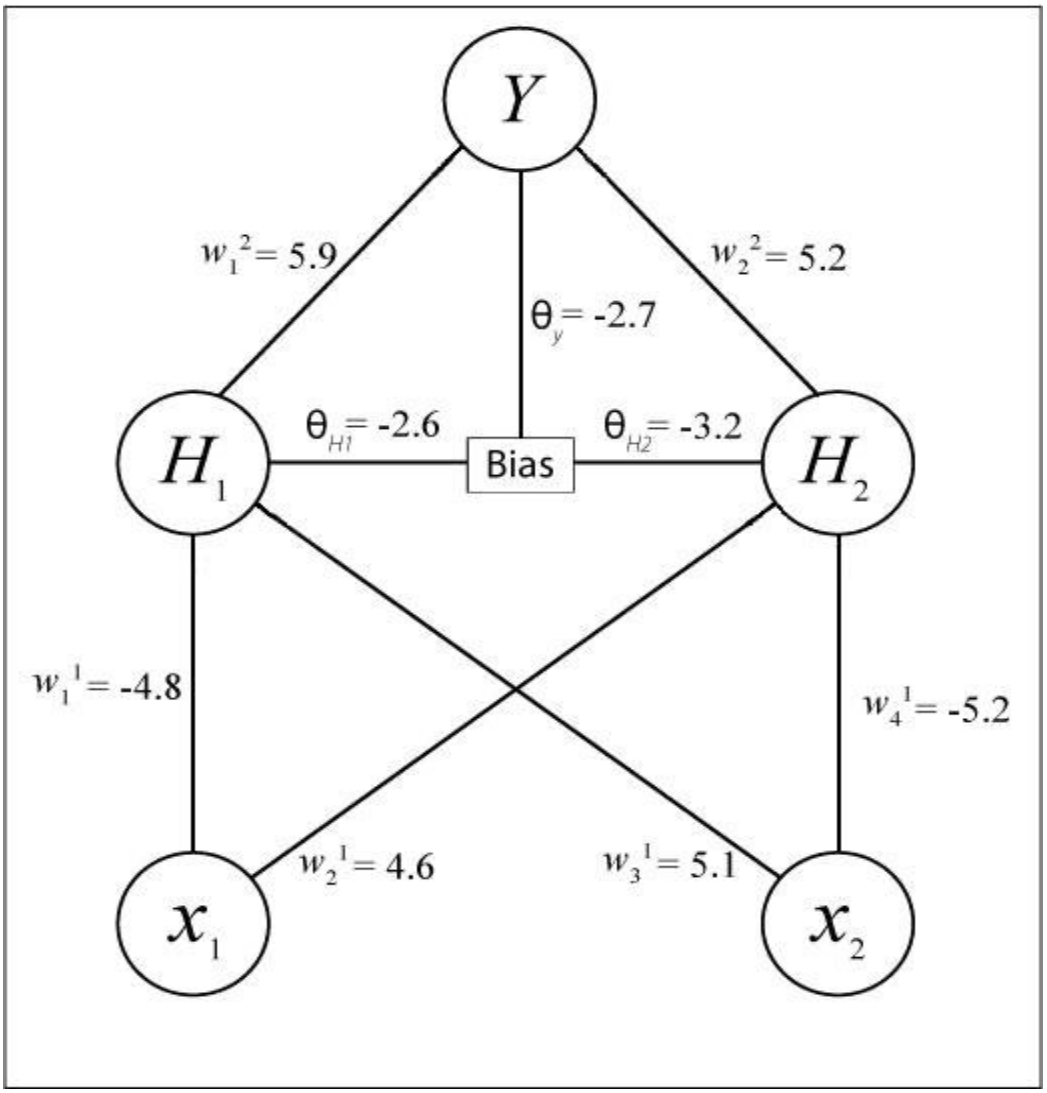
Example: BP Algorithm for Exclusive OR Problem



Given the minimum error $E_{\min} = 0.0001$, find the network output for the XOR data

Input		Actual Output	MLP Network Output (Y)	BP Network Output (Y)
0	1	1	0.9401 (*E=0.0018)*	0.9972 (*E=0.000004)*
1	0	1		
0	0	0		
1	1	0		

$$Y = f\left(net\right) = 1/\left(1+e^{-net}\right)$$
$$net = \sum w_i^j x_i + \theta_j$$

Generally,

$$net_{H_1} = \sum_{i=1}^{2} x_i w + \theta_{H_1},$$

$$net_{H_1} = x_1 w_1^1 + x_2 w_3^1 + \theta_{H_1},$$

$$net_{H_1} = 0*\left(-4.8\right)+1*\left(5.1\right)+\left(-2.6\right) = 2.5$$

$$H_1 = f\left(net_{H_1}\right) = \frac{1}{1+e^{-2.5}} = 0.9241$$

Thus, output for hidden nodes H_1 = 0.9241

Output for hidden node H_2,

$$net_{H_2} = \sum_{i=1}^{2} x_i w + \theta_{H_2},$$

$$net_{H_2} = x_1 w_2^1 + x_2 w_4^1 + \theta_{H_2},$$

$$net_{H_2} = 0*\left(4.6\right)+1*\left(-5.2\right)+\left(-3.2\right) = -8.4$$

$$H_2 = f\left(net_{H_2}\right) = \frac{1}{1+e^{-(-8.4)}} \approx 0.00022$$

Thus, output for hidden nodes H_2 = 0.00022

Calculate the weighted sum going into the output neuron:

$$net_Y = \sum_{i=1}^{2} H_i w + \theta_Y ,$$

$$net_Y = H_1 w_1^2 + H_2 w_2^2 + \theta_Y ,$$

$$net_Y = 0.9241 * (5.9) + 0.00022 * (5.2) + (-2.7) = 2.7533$$

$$Y = f(net_Y) = \frac{1}{1 + e^{-(2.7533)}} \approx 0.9401$$

Mean-squared error is used to quantify the error of the network:

$$err = 0.5 * (\text{Target Output - Actual Output})^2$$

$$err = 0.5 * (1 - 0.9401)^2 \approx 0.0018$$

Next, apply the back-propagation algorithm to reduce the error:

$$\delta_k = o_k (1 - o_k)(t_k - o_k)$$

$$\delta_k = (Actual\ Output - Network\ Output) * Network\ Output * (1 - Network\ Output)$$

Calculate output node error δ_Y :

$$\delta_Y = (1.0 - 0.94) * 0.94 * (1 - 0.94)$$

$$= 0.06 * 0.94 * 0.06 = 0.003384$$

To calculate an error for hidden nodes, we need to use the derivative of sigmoid equation:

$$f'(x) = x * (1.0 - x)$$

The sigmoidal function is useful in that it can be differentiated. Its output values are restricted to be between $[0.0, 1.0]$. This is useful for the classification purposes.

Error for hidden nodes δ_j:

$$\delta_j = o_j \left(1 - o_j\right) \sum \delta_k w_{kj},$$

$$\delta_{H_2} = \left(\delta_Y * w_2^2\right) * H_2 * \left(1.0 - H_2\right)$$

$$= \left(0.003384 * 5.2\right) * 0.00022 * \left(1.0 - 0.00022\right)$$

$$= 0.0175968 * 0.00022 * 0.99978$$

$$= 0.00000387$$

$$\delta_{H_1} = \left(\delta_Y * w_1^2\right) * H_1 * \left(1.0 - H_1\right)$$

$$= \left(0.003384 * 5.9\right) * 0.924 * \left(1.0 - 0.924\right)$$

$$= 0.0199656 * 0.924 * 0.076$$

$$= 0.001402064$$

The weight adaptation formulations are given as below:

1. Between the hidden layer (*j*) and output layer (*k*),

   The change of weight is given as:

$$\Delta w_{kj}\left(t+1\right) = \eta\ \delta_k\ o_j\ + \alpha\ \Delta w_{kj}\left(t\right),$$

   where

   η is a learning rate that affects the convergence speed and stability of the weights during learning, and

   α is a momentum term to speed up te MLP propagation training, $\alpha \in [0.1, 1.0]$

   Thus, the new adjusted weights are given as,

$$w_{kj}\left(t+1\right)=w_{kj}\left(\mathrm{t}\right)+\Delta w_{kj}\left(t+1\right),$$

By obtaining the error values for the output and hidden layers, adjustment of the weights can be done. Using learning rate (η) of 0.5:

$$w_2^2=w_2^2+\left(\eta*\delta_Y*H_2\right)$$

$$=5.2+\left(0.5*0.003384*0.00022\right)$$

$$=5.200000372$$

$$w_1^2=w_1^2+\left(\eta*\delta_Y*H_1\right)$$

$$=5.9+\left(0.5*0.003384*0.924\right)$$

$$=5.901563408$$

Update output bias:

$$\theta_Y=\eta*\delta_Y*\theta_Y$$

$$=0.5*0.00384*-2.7$$

$$=-0.0045684$$

Next, calculate adjustment of the hidden weights (from input to hidden nodes):

2. Between the input layer (*i*) and hidden layer (*j*),

The change of weight is given as:

$$\Delta w_{ji}\left(t+1\right)=\eta\ \delta_j\ o_i\ +\alpha\ \Delta w_{ij}\left(t\right).$$

Thus, the new adjusted weights are given as,

$$w_{ij}\left(t+1\right)=w_{ij}\left(\mathrm{t}\right)\ +\Delta w_{ij}\left(t+1\right),$$

where,

$$\delta_j=o_j\left(1-o_j\right)\ \sum\delta_k w_{kj},$$

$$o_k=f\left(net_k\right)=\frac{1}{1+e^{-net_k}},$$

and

$$net_k=\sum_k w_{kj}\ o_j+\theta_k$$

Thus,

$$w_4^1 = w_4^1 + \left(\eta * \delta_{H_2} * x_2\right)$$

$$= -5.2 + \left(0.5 * 0.00000387 * 1\right)$$

$$= -5.19998065$$

$$w_2^1 = w_2^1 + \left(\eta * \delta_{H_2} * x_1\right)$$

$$= 4.6 + \left(0.5 * 0.00000387 * 0\right)$$

$$= 4.6$$

$$w_3^1 = w_3^1 + \left(\eta * \delta_{H_1} * x_2\right)$$

$$= 5.1 + \left(0.5 * 0.001402064 * 1\right)$$

$$= 5.100701032$$

$$w_1^1 = w_1^1 + \left(\eta * \delta_{H_1} * x_1\right)$$

$$= -4.8 + \left(0.5 * 0.001402064 * 0\right)$$

$$= -4.8$$

@Copyright
Prof. Dr. Siti Mariyam Shamsuddin
UTM Big Data Centre
2nd October 2017

Update the cell biases:

$$\theta_{H_2} = \eta * \delta_{H_2} * \theta_{H_2}$$

$$= 0.5 * 0.00000387 * -3.2$$

$$= -0.000006192$$

$$\theta_{H_1} = \eta * \delta_{H_1} * \theta_{H_1}$$

$$= 0.5 * 0.001402064 * -2.6$$

$$= -0.001822683$$

To verify, calculate the feed-forward algorithm:

$$net_{H_1} = x_1 w_1^1 + x_2 w_3^1 + \theta_{H_1},$$

$$net_{H_1} = 0*(-4.8) + 1*(5.100701032) + (-0.001822683)$$

$$= 5.098878349$$

$$H_1 = f\left(net_{H_1}\right) = \frac{1}{1+e^{-5.098878349}} = 0.993933438$$

$$net_{H_2} = x_1 w_2^1 + x_2 w_4^1 + \theta_{H_2},$$

$$net_{H_2} = 0*(4.6) + 1*(-5.19998065) + (-0.000006192)$$

$$= -5.199974458$$

$$H_2 = f\left(net_{H_2}\right) = \frac{1}{1+e^{-(-5.199974458)}} = 0.005486438$$

$$net_Y = H_1 w_1^2 + H_2 w_2^2 + \theta_Y,$$

$$net_Y = 0.993933438*(5.901563408) + 0.005486438*(5.200000372) + (-0.0045684)$$

$$= 5.889722288$$

$$Y = f\left(net_Y\right) = \frac{1}{1+e^{-(5.889722288)}} \approx 0.9972$$

@Copyright
Prof. Dr. Siti Mariyam Shamsuddin
UTM Big Data Centre
2nd October 2017

The error value of the network:

$$err = 0.5 * \left(Actual\ Output - Network\ Output \right)^2$$

$$err = 0.5 * \left(1 - 0.9972 \right)^2 \approx 0.000004$$